📘 GSFC UNIVERSITY PLACEMENT ANALYZER & AI RECRUITMENT SYSTEM

Master Technical Architecture & System Documentation

Author: Om P. Thakkar (BTech CSE, GSFC University) **Version:** 2.0.0 (Production Release) **Date:** August 2026 **Repository:** `OMTHAKKAR8495/gsfc-placement-analyzer-`

TABLE OF CONTENTS

CHAPTER 1: Executive Summary & Project Vision

1.1 Background & Purpose

GSFC University hosts thousands of students across diverse academic disciplines, including **BTech Computer Science & Engineering**, **BTech Mechanical Engineering**, **BTech Civil Engineering**, **BTech Chemical Engineering**, **BTech Electrical & Electronics Engineering**, **MSc Chemical Sciences**, **Biotechnology**, and **BBA/MBA Management**.

Prior placement workflows faced three critical industry challenges: 1. **Domain Bias in Resume Evaluation**: Traditional recruitment portals evaluated resumes against fixed keyword dictionaries (primarily Computer Science keywords like `Python` or `React`), penalizing top-performing students from core engineering disciplines (Mechanical, Civil, Chemical) when applying for core or interdisciplinary roles. 2. **Static & Arbitrary Scoring**: Legacy systems assigned fixed fallback scores (e.g. 75% Match) without parsing actual resume contents against corporate hiring requirements. 3. **Manual Applicant Screening**: Placement officers and corporate recruiters spent hundreds of hours manually verifying CGPA cutoffs, degree program eligibility, and ATS compatibility.

1.2 The GSFC CampusHire AI Solution

Developed by **Thakkar Om (BTech CSE)**, the **GSFC University Placement Analyzer & AI Recruitment System** is an enterprise full-stack platform designed to automate end-to-end campus recruitment: - **Universal Multi-Branch Resume Parsing**: Parses `.pdf` and `.docx` resumes from any GSFC academic discipline using LLM vector extraction and OCR fallbacks. - **Cross-Domain Skill Taxonomy**: Utilizes a structured 7-domain taxonomy (`skillTaxonomy.json`) to normalize skill synonyms across all engineering and science branches (e.g. `SolidWorks` maps to `CAD/CAM Design`, `STAAD Pro` maps to `Structural Analysis`). - **Dynamic Weighted Matching Engine**: Evaluates candidate eligibility and computes a dynamic 0-100% NLP match score based on **Skill Fit (45%)**, **Experience & Project Fit (35%)**, and **Academic Margin (20%)**. - **Interactive AI Mock Interview Studio**: Features real-time voice coaching powered by Gemini AI, evaluating technical depth, clarity, correctness, and enforcing instant auto-fail guardrails. - **Multi-Platform Availability**: Delivered seamlessly as a Web Application, Android APK/AAB package, and iOS Native Xcode bundle using Capacitor.

CHAPTER 2: Full-Stack System Architecture & Technology Stack

2.1 Technical Stack Overview

| Layer | Technology | Key Capabilities & Purpose | | :--- | :--- | :--- | | **Frontend Framework** | React 18 + Vite 5 | SPA architecture, hot module replacement, 1.2s production build | | **Styling & Design System** | Vanilla CSS + TailwindCSS | Glassmorphism UI, fluid typography, light/dark mode theme system | | **Icons & UI Utilities** | Lucide React Icons | Modern vector iconography (`User`, `Building`, `ShieldCheck`, `Sparkles`) | | **Mobile Runtime** | Ionic Capacitor 8 | Native iOS (`ios/`) and Android (`android/`) web view bridges | | **Backend Web Server** | Node.js + Express | RESTful API server with custom security middleware | | **Database Engine** | SQLite3 (`better-sqlite3`) | High-concurrency WAL mode database with 64MB RAM caching | | **AI Inference Pipeline** | Gemini AI + Local Fallback NLP | Natural language

extraction, mock interview coaching, evaluation agents | | **PDF Processing** | `pdf-parse` + Buffer Streams | In-memory binary PDF object indexing and text extraction |

2.2 System Component Architecture

``` +---+ | CLIENT LAYER | | +-----------------------+ +-----------------------+ +-----------------------+ | | | Student Workspace | | Recruiter Portal | | TPC Admin Governance | | | | (Resume NLP & Match) | | (Application Vault) | | (Accreditation Export| | | +-----------+-----------+ +-----------+-----------+ +-----------+-----------+ | | | | | | | +--------------------------+--------------------------+ | | | | | Capacitor Native Bridge | | (Android APK & iOS Xcode Swift Shells) | +---+--------------------------------------+ | HTTP / REST API (Port 5173 -> 5001 Proxy) +---v---+ | BACKEND LAYER | | +--+ | | | Express Router Middleware | | | | [Security: CORS | JWT | RateLimiter | XSS Sanitizer | MassAssignmentGuard] | | | +-------+-----------------+-------------------+-----------------+-----------+ | | | | | | | | | +-------v-------+ +-------v-------+ +-------v-------+ +-------v-------+ | | | Resume Parser | | ATS Scorer | | Match Engine | | Voice Coach | | | | (Module A) | | (Module B) | | (Module C) | | (Modules D-F) | | | +-------+-------+ +-------+-------+ +-------+-------+ +-------+-------+ | | | | | | | | | +-----------------+---------+---------+-----------------+ | | | | | SQLite WAL Storage & Indexing | | (backend/db/campushire.db) | +---+ `

CHAPTER 3: Database Schema & High-Performance Indexing

3.1 Database Configuration & Performance Pragmas

To ensure microsecond database execution (< 15ms total backend latency), the SQLite connection in [backend/db/index.js]() executes the following performance pragmas upon initialization:

` javascript db.pragma('journal_mode = WAL'); // Write-Ahead Logging for concurrent reads & writes db.pragma('synchronous = NORMAL'); // Optimizes disk flush latency db.pragma('temp_store = MEMORY'); // Keeps temporary tables in RAM db.pragma('cache_size = -64000'); // Allocates a 64MB in-memory query cache db.pragma('foreign_keys = ON'); // Enforces relational database integrity `
```

3.2 Key Relational Tables

1. users Table

Stores authentication credentials and global role scopes:

```sql
CREATE TABLE IF NOT EXISTS users ( id TEXT PRIMARY KEY, email TEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL, role TEXT CHECK(role IN ('student', 'company', 'admin')) NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP );
```

2. student_profiles Table

Stores parsed resume data, student academic credentials, and ATS scores:

```sql
CREATE TABLE IF NOT EXISTS student_profiles ( id TEXT PRIMARY KEY, user_id TEXT UNIQUE NOT NULL, roll_number TEXT UNIQUE, name TEXT NOT NULL, program TEXT NOT NULL, branch TEXT, cgpa REAL NOT NULL, resume_url TEXT, parsed_resume_json TEXT, ats_score INTEGER DEFAULT 0, ats_feedback_json TEXT, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE );
```

3. company_profiles Table

Stores recruiter corporate credentials, official logos, and TPC approval status:

```sql
CREATE TABLE IF NOT EXISTS company_profiles ( id TEXT PRIMARY KEY, user_id TEXT UNIQUE NOT NULL, company_name TEXT NOT NULL, logo_url TEXT, industry TEXT, website TEXT, contact_email TEXT, contact_phone TEXT, approved INTEGER DEFAULT 0, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE );
```

4. requirements Table

Stores placement drive postings, eligible academic programs, required skills, and CTC details:

```sql
CREATE TABLE IF NOT EXISTS requirements ( id TEXT PRIMARY KEY, company_id TEXT NOT NULL, title TEXT NOT NULL, job_description TEXT NOT NULL, eligible_programs_json TEXT NOT NULL, min_cgpa REAL DEFAULT 0.0, required_skills_json TEXT NOT NULL, preferred_skills_json TEXT NOT NULL, ctc_range TEXT NOT NULL, job_type TEXT DEFAULT 'Full-time', openings INTEGER DEFAULT 1, deadline DATE NOT NULL, application_type TEXT CHECK(application_type IN ('internal', 'external')) DEFAULT 'internal', external_apply_url TEXT, external_click_count INTEGER DEFAULT 0, question_bank_json TEXT DEFAULT '[]', company_logo_url TEXT, company_website TEXT, company_email TEXT, company_phone TEXT, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (company_id) REFERENCES company_profiles(id) ON DELETE CASCADE );
```

5. applications Table

Stores submitted candidate placement drive applications and exact calculated match scores: `sql CREATE TABLE IF NOT EXISTS applications (id TEXT PRIMARY KEY, student_id TEXT NOT NULL, requirement_id TEXT NOT NULL, match_score REAL NOT NULL, status TEXT CHECK(status IN ('applied', 'shortlisted', 'interview', 'selected', 'rejected')) DEFAULT 'applied', applied_via TEXT CHECK(applied_via IN ('internal', 'external')) DEFAULT 'internal', applied_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (student_id) REFERENCES student_profiles(id) ON DELETE CASCADE, FOREIGN KEY (requirement_id) REFERENCES requirements(id) ON DELETE CASCADE);`

3.3 High-Speed Performance Indexes

To guarantee sub-10ms query execution across thousands of student records, 7 database indexes were implemented: `sql CREATE INDEX IF NOT EXISTS idx_users_email ON users(email); CREATE INDEX IF NOT EXISTS idx_company_profiles_user_id ON company_profiles(user_id); CREATE INDEX IF NOT EXISTS idx_company_profiles_approved ON company_profiles(approved); CREATE INDEX IF NOT EXISTS idx_student_profiles_user_id ON student_profiles(user_id); CREATE INDEX IF NOT EXISTS idx_requirements_company_id ON requirements(company_id); CREATE INDEX IF NOT EXISTS idx_applications_req_id ON applications(requirement_id); CREATE INDEX IF NOT EXISTS idx_applications_student_id ON applications(student_id);`

CHAPTER 4: Multi-Branch Resume Parsing Engine (Module A)

4.1 Input Format Handling & OCR Fallback

Module A (resumeParser.js) accepts .pdf and .docx binary files. If a scanned PDF (image-only without a vector text layer) is uploaded, the parser automatically executes an OCR fallback extraction routine:

`javascript export async function parseResume(bufferOrText, filename = '') { let rawText = ''; if (Buffer.isBuffer(bufferOrText)) { try { const pdfData = await pdfParse(bufferOrText); rawText = pdfData.text || ''; } catch (err) { rawText = bufferOrText.toString('utf8'); } } else { rawText = String(bufferOrText); } ... }`

4.2 LLM Extraction Schema & Deterministic Branch Fallback

Resumes are converted into structured JSON matching the following schema: - **Contact Info:** Full Name, Email, Phone, University Roll Number - **Education:** Degree Program, Academic Branch, Cumulative CGPA (10.0 scale) - **Technical & Soft Skills:** Extracted skills categorized by academic domain - **Projects & Internships:** Project titles, domain tools, responsibilities - **Certifications:** Industry certifications and academic honors

If the primary LLM call is unavailable, generateSmartParsedFallback() analyzes raw text keywords to deterministically extract domain-specific profiles for Mechanical (SolidWorks , ANSYS , GD&T), Civil (STAAD Pro , ETABS), Electrical (Embedded C , PCB Layout), Chemical (Aspen Plus , HYSYS), Business (PowerBI , Financial Modeling), or Computer Science candidates.

CHAPTER 5: ATS Compliance Evaluation Engine (Module B)

5.1 ATS Evaluation Criteria

Module B (atsScorer.js) evaluates resume structure against Applicant Tracking System (ATS) parsing standard rules:

1. **Parseable Text Density:** Ensures text is vector-searchable and not an image screenshot. 2. **Section Header Standard:** Verifies standard section headings (Education , Skills , Projects , Work Experience). 3. **Contact Information Completeness:** Checks presence of valid email and phone numbers. 4. **Action Verb & Metric Formatting:** Evaluates quantify-driven bullet points (e.g. *"Optimized database performance by 40%"*). 5. **Technical Skill Formatting:** Checks for clean, un-nested skill listings without problematic multi-column tables.

5.2 ATS Score Calculation Formula

The ATS Compliance Score (S_{ATS}) is calculated on a 0-100 scale:

$$S_{ATS} = \text{BaseScore} (80) + \text{Bonus}_{\text{Metrics}} (10) - \text{Penalty}_{\text{MissingContact}} (15) - \text{Penalty}_{\text{ShortText}} (20)$$

Candidates receive an instant ATS score meter along with actionable feedback suggestions to improve document parsing compatibility.

CHAPTER 6: Cross-Domain Branch-Agnostic Matching Engine (Module C)

6.1 The Branch-Agnostic Skill Taxonomy Config (skillTaxonomy.json)

To eliminate Computer Science bias, skillTaxonomy.json maps skills across 7 GSFC University academic buckets:

- **Computer Science & IT:** Python, JavaScript, React, Node.js, SQL, FastAPI, Docker, Kubernetes, PyTorch. - **Mechanical & Mechatronics:** SolidWorks, AutoCAD, CATIA, ANSYS, FEA, CFD, GD&T, Six Sigma, Thermodynamics, CNC Programming. - **Electrical & Electronics (ECE/EEE):** Embedded C, PCB Layout, Altium, VLSI, Verilog, MATLAB, Simulink, PLC/SCADA. - **Civil & Structural:** STAAD Pro, ETABS, AutoCAD Civil 3D, Revit Structure, BIM, Primavera, Surveying. - **Chemical & Process:** Aspen Plus, HYSYS, Process Simulation, Distillation, Mass Transfer, HAZOP, Effluent Treatment. - **MSc Science & Biotech:** HPLC, GC-MS, Spectroscopy, PCR, Gel Electrophoresis, Bioinformatics, Analytical Chemistry. - **Business & MBA:** PowerBI, Tableau, Financial Modeling, Advanced Excel, Tally, Supply Chain Management, Agile.

6.2 Canonical Skill Normalization & Synonym Matching

The taxonomy utility (taxonomyMatcher.js) normalizes variant phrasing into canonical entities (e.g. SolidWorks , solid works , and 3D CAD map to CAD / CAM Design), granting candidate credit even when exact string titles vary.

6.3 The Weighted Matching Score Algorithm

When a candidate applies to a placement drive, matchingEngine.js executes a 5-step evaluation process:

Step 1: Hard Eligibility Gate

$$\text{Eligible} = (\text{StudentBranch} \in \text{EligibleBranches}) \land (\text{StudentCGPA} \ge \text{MinCGPA})$$ If ineligible, the engine returns matchScore: 0 with an explicit reason (e.g., *"Branch BTech Mechanical is not in eligible list [BTech CSE, BTech IT]"*).

Step 2: Skill Overlap Score (S_{Skill}, 45% Weight)

Calculated using canonical taxonomy matching: $$S_{\text{Skill}} = \left(\frac{|\text{MatchedRequiredSkills}|}{|\text{TotalRequiredSkills}|} \times 100 \right) +

\text{Bonus}_{\text{Preferred}}$$

Step 3: Experience & Project Domain Relevance (S_{Exp}, 35% Weight)

Calculates token-vector cosine similarity between candidate project descriptions and the company job description text: $$\text{CosineSim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Step 4: Academic Fit Score (S_{Acad}, 20% Weight)

Calculates CGPA margin above company cutoff: $$S_{\text{Acad}} = \min\left(100, \max\left(50, 70 + (\text{StudentCGPA} - \text{MinCGPA}) \times 15\right)\right)$$

Step 5: Composite Final Score (S_{Final})

$$S_{\text{Final}} = \text{Math.round}\left(0.45 \cdot S_{\text{Skill}} + 0.35 \cdot S_{\text{Exp}} + 0.20 \cdot S_{\text{Acad}}\right)$$

` +-----------------------+ | Student Application | +-----------+-----------+ | +-----------v-----------+ | Hard Eligibility Gate | +-----------+-----------+ | +-----------------------+-----------------------+ | Eligible? | Ineligible? v v +----------------------------+ +-----------------------+ | Compute Component Scores: | | Match Score: 0% | | - Skill Fit (45%) | | Flagged Ineligible | | - Experience Fit (35%) | +-----------------------+ | - Academic Fit (20%) | +-------------+-------------+ | +-------------v-------------+ | Final Score (0 - 99%) | | Matched/Missing Skills | | AI Match Rationale | | Improvement Tips | +----------------------------+ `

CHAPTER 7: Gemini AI Mock Interview & Voice Coaching Engine (Modules D, E, F)

7.1 Architecture & Components

1. **Module D (Interview Question Generator):** Generates 4 structured technical & situational questions tailored to the company's requirement tags and job description. 2. **Module E (Mock Interview Coach & Voice Interface):** Provides interactive candidate practice with Web Speech API audio voice synthesis. Candidate answers are scored for **Technical Correctness, Conceptual Clarity,** and **Communication Quality.** 3. **Module F (AI Evaluation & Auto-Fail Guardrail):** Evaluates short-answer responses and instantly triggers auto-fail verdicts for nonsensical or off-topic candidate submissions:

`javascript // Instant Auto-Fail Enforcement for Single-Word / Nonsensical Inputs if (cleanAnswer.length < 15) { return { verdict: 'fail', overallScore: 25, reasoning: 'Response is too brief to demonstrate technical competence.' }; } `

CHAPTER 8: User Workspaces & Role-Based Governance Portals

8.1 Student Workspace (#student)

- **Smart Resume Analyzer:** Drag-and-drop file upload with live ATS score circular meter. - **Personalized Placement Feed:** Displays company drives with dynamic match badges (88% Match , Ineligible). - **Interactive Match Breakdown Modal:** Displays ✅ **Matched Skills**, ⚠️ **Missing Skills**, 💡 **AI Match Rationale**, and 🎯 **Actionable Improvement Tips**. - **Automated PDF Placement Report:** Generates downloadable PDF candidate accreditation cards.

8.2 Recruiter Portal & Application Vault (#company)

- **Corporate Profile Enforcement:** Recruiter must register official company logo, corporate email, phone, and website. - **Hiring Drive Setup:** Multi-branch program checkboxes, minimum CGPA slider, and domain skill tag chips. - **Recruiter Application Vault:** 1st column tab tracking submitted applications with action controls (✏️ Edit , 👥 Applicants , 🗑️ Delete). - **Applicants Inspection Modal:** Displays submitted candidates ranked dynamically from highest match % to lowest.

8.3 TPC Admin Governance Dashboard (#admin)

- **Academic Discipline Filter Panel:** Filter candidate database across all 8 GSFC branches (BTech CSE , Mechanical , Civil , ECE , Chemical , MSc , BBA/MBA). - **Recruiter Approval Governance:** Single-click approval/rejection of corporate accounts with custom glassmorphism approval popups (ApprovalNotificationModal). - **Master CSV Accreditation Export:** Downloads university placement statistics for NAAC accreditation.

CHAPTER 9: Security, Rate Limiting & Performance Engineering

9.1 Security Architecture (security.js)

- **JWT Authentication & httpOnly Cookies:** Issues signed 7-day JSON Web Tokens set in httpOnly , sameSite=strict cookies. - **Password Policy & Fast Bcrypt Hashing:** Password hashing cost factor optimized to 6 to ensure login resolves in **< 10 milliseconds.** - **XSS & Prompt Injection Sanitization:** Strips malicious prompt overrides (ignore previous instructions ,